DENTAL X-RAY IMAGE CLASSIFICATION USING DEEP LEARNING

A MINI PROJECT REPORT SUBMITTED TO

Swarnandhra College of Engineering and Technology

in partial fulfillment of the requirements for the award of the degree

MASTER OF COMPUTER APPLICATIONS

By

A. TEJA VENKATA RAJU
(24A21F0072)

Under the guidance of

Mr. A.N.L. KUMAR
Assosiate Professor

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS
SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
(Approved by AICTE & Permanently Affiliated to JNTUK)
(Accredited by NAAC with "A" Grade)
SEETHARAMPURAM, NARASAPUR-534280

2024-2026

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY

(AUTONOMUS)

Seetharampuram, Narasapur-534 280.

CERTIFICATE

This is to certify that this project work entitled **"DENTAL X-RAY IMAGE CLASSIFICATION USING DEEP LEARNING"** is the Bonafide work of **Mr.A.TEJA VENKATA RAJU** who carried out the work under my supervision, and submitted in partial fulfillment of the requirements for the award of the degree, **Master of Computer Applications**, during the academic year **2024-2026.**

Project Coordinator

Dr.B.GOHIN

Associate Professor

Project Supervisor

Mr. A.N.L. KUMAR

Assosiate Professor

Head of Department

Mr. A.N.L. KUMAR

Head Of the Department of MCA

DECLARATION

I certify that

The work contained in the project work is original and has been done by me under the guidance of my supervisor The work has not been submitted to any other University for the award of any degree or diploma. The guidelines of the University are followed in writing this report.

Date:

A.TEJA VENKAT RAJU
(24A21F0072)

ACKNOWLEDGEMENT

I extend my heartfelt gratitude to the Almighty for giving me strength in proceeding with this project title "**DENTAL X-RAY IMAGE CLASSIFICATION USING DEEP LEARNING".**

I express my heartfelt gratitude to my parents for supporting me in all the ways in every walk of my life.

I express my sincere thanks to Honorable **Sri. K.V. Satayanarayana,** Chairman of our college for making necessary arrangement for doing the project.

I wish to express my gratitude to **Dr. S. Suresh Kumar,** Principal of our college, for giving us permission to carry out this project.

I express my sincere thanks to **Mr. A.N.L. Kumar,** Associate Professor, Head of the Department of M.C.A. for his learned suggestions and encouragement which made this project a successful one.

I convey my sincere thanks to my project guide **Mr. A.N.L. Kumar,** Associate Professor, HOD,Department of MCA for his learned suggestions and encouragement which made this project a success.

I express my sincere thanks of all the faculty members of our Department of MCA for their valuable support throughout this project work.

I wish to express my thanks to my friends for their enthusiasm, support and encouragement for the completion of this project.

A.TEJA VENKATA RAJU

Regd. No.24A21F0072

DENTAL X-RAY IMAGE CLASSIFICATION USING DEEP LEARNING

ABSTRACT

A robust deep learning pipeline was developed to classify dental X-ray images and detect common dental abnormalities such as cavities, fractures, crowns, fillings, and healthy teeth.We collected and curated a large labeled dateset of dental radiography, annotated by dental professionals to ensure high-quality ground truth. Key input features included raw image pixels, extracted texture descriptors, localized ROI measurements, and patient metadata (age, tooth position) after careful anonymization.

Image preproessing steps — contrast enhancement, noise reduction, histogram equalization, and data augmentation (rotation, flipping, scaling) — improved model generalization.Feature engineering used convectional feature extractors and handcrafted descriptors to capture both global and local patterns in X-rays.
Because abnormal cases were far fewer than healthy ones, we applied oversampling (SMOTE for tabular features), class weighting, and careful augmentation to mitigate class imbalance.

We trained and compared multiple models: logistic regression and decision trees on engineered features, CNN s for end-to-end image learning, and object-detection architectures(e.g.,YOLOv8)for lesion localization.Model selection prioritized clinically meaningful metrics — precision, recall (sensitivity), specificity, and F1-score — to balance catching true positives while minimizing false alarms.Cross-validation and stratified holdout testing ensured robust evaluation across patient subsets and imaging devices.Explain-ability techniques (Grad-CAM, salience maps) were integrated so clinicians could verify which image regions informed each prediction.The deep convectional models and YO LO-style detector significantly outperformed classical methods in both classification and localization tasks.Top models achieved overall ac-curacies above 95%, with the best neural network reaching sensitivity and F1-scores that are clinically actionable for screening.Deployment considerations included model pruning, quantization, and a lightweight inference pipeline to enable near real-time analysis in clinic environments.

Index

S.No	Topic	Page No
1	Chapter 1: Introduction	1
	1.1 Project Overview	2
	1.2 Problem Statement	2
	1.3 Project Objectives	3
	1.4 Project Scope	3
2	Chapter 2: Literature Survey	4
	2.1 Existing System Analysis	4
	2.2 Research Papers Review	4
	2.3 Similar Systems Study	4
	2.4 Technology Review	5
3	Chapter 3: System Analysis	6
	3.1 Requirement Gathering	6
	3.2 Feasibility Study	6
	3.2.1 Technical Feasibility	6
	3.2.2 Economic Feasibility	7
	3.2.3 Operational Feasibility	7
	3.3 SWOT Analysis	7
4	Chapter 4: System Requirements Specification	8
	4.1 Functional Requirements	8
	4.2 Non-Functional Requirements	8
	4.3 Hardware Requirements	10
	4.4 Software Requirements	11

	4.5 Network Requirements	12
5	Chapter 5: System Design	13
	5.1 Architecture Design	13
	5.2 UML Diagrams	14
	5.2.1 Use Case Diagrams	14
	5.2.2 Class Diagrams	16
	5.2.3 Sequence Diagrams	17
	5.2.4 Activity Diagrams	17
	5.3 Database Design	18
	5.3.1 ER Diagrams	19
	5.3.2 Data Dictionary	20
6	Chapter 6: Implementation	21
	6.1 Algorithms	21
	6.2 Pseudocode	22
	6.3 Source Code	24
	6.4 Database Queries	32
7	Chapter 7: Testing	33
	7.1 Test Plan	33
	7.2 Test Cases	35
	7.2.1 Unit Testing	35
	7.2.2 Integration Testing	37
	7.2.3 System Testing	38

	7.3 Test Results	39
8	Chapter 8: Implementation Results	40
	8.1 Screenshots	40
	8.2 Output Screens	42
	8.3 Performance Metrics	45
9	Chapter 9: Conclusion and Future Scope	46
	9.1 Conclusion	46
	9.2 Limitations	47
	9.3 Future Enhancements	47
10	References	50
11	Appendices	51
	Appendix A: User Manual	51
	Appendix B: Installation Guide	51

1. Introduction

The project aims to build an intelligent automated system capable of analyzing dental X-ray images using deep learning techniques. It reduces the need for complete manual inspection by dentists and minimizes the chances of human error during diagnosis. By using advanced deep learning, the system can understand patterns in X-ray images and accurately detect dental issues. It identifies conditions such as cavities, crowns, fillings, fractures, and other abnormalities present in the teeth. The YO LO-based model processes each image efficiently and highlights the affected areas clearly. This automated approach ensures faster diagnostic results compared to traditional methods. It also improves the accuracy and reliability of dental analysis. The system provides consistent predictions, regardless of the number of images processed. It helps dentists make better decisions with the support of AI-generated insights. Early detection of dental problems becomes easier with this model. The system is designed to enhance productivity in dental clinics. It serves as a useful tool for medical professionals and researchers. The overall goal is to integrate artificial intelligence into dental imaging. Ultimately, this project demonstrates how machine learning can transform modern dental care.

A YOLOv8 model, which is widely known for its high speed and accuracy in image detection, is used in this project to analyze dental X-ray images. The model is trained using a properly labeled dental data set that contains different types of dental conditions. Through this training, YOLOv8 learns to identify and differentiate abnormalities such as cavities, crowns, fillings, fractures, and even healthy teeth. It processes the features present in each X-ray image and understands the patterns associated with different dental issues. Once the image is analyzed, the model is able to classify the specific condition it detects. It also highlights the affected regions by drawing bounding boxes around them. This helps in providing a clear visual indication of the problem areas. The system ensures that each detection is fast and precise. By combining deep learning and image processing, the model provides consistent diagnostic support. Its ability to handle complex X-ray patterns makes it suitable for dental analysis. The automated classification saves time for dental professionals. It enhances the accuracy of dental diagnosis. Overall, YOLOv8 plays a crucial role in making the system reliable and effective.

This automated approach greatly improves the efficiency of dental diagnosis by delivering faster and more reliable results. It reduces the chances of human error that often occur during manual examination of X-ray images. The system ensures that every prediction is consistent, regardless of how many images are processed. By providing accurate identification of dental abnormalities, it supports dentists in making better clinical decisions. The model's quick response time helps in speeding up the overall diagnostic workflow. Its ability to analyze complex X-ray features enhances the clarity of findings. Dental professionals can rely on the system for precise detection of issues. This reduces the workload on dentists and assists them in critical evaluations. The automated predictions act as a second opinion for improved accuracy. It also helps in early identification of dental problems. The system brings more confidence to treatment planning. Patients receive faster feedback about their dental health. The model contributes to maintaining high diagnostic standards. Overall, it strengthens decision-making in dental care. This technology marks a significant advancement in AI-supported dental diagnosis.

1.1 Project Overview

This project focuses on automatically classifying dental X-ray images into different categories such as cavities, fillings, crowns, fractures, and healthy teeth using advanced machine learning techniques. A YOLOv8 model is utilized due to its high speed and accuracy in detecting image features. The model is trained on a labeled dental data set, allowing it to learn the unique patterns associated with various dental conditions. By analyzing the X-ray images, it identifies abnormalities with precision. The automated detection process helps in reducing manual effort during diagnosis. It also improves the consistency and reliability of results. The system provides clear labeling for each detected condition. This helps dental professionals make quicker decisions. Overall, the model plays a crucial role in simplifying and enhancing dental X-ray classification.

1.2 Problem Statement

Manual diagnosis of dental X-rays can often be slow, time-consuming, and prone to human error. Dentists may overlook small abnormalities due to fatigue or

image complexity. To overcome these challenges, there is a strong need for an automated system that can support the diagnostic process. Such a system can analyze X-ray images quickly and with greater consistency. By reducing human error, it improves the overall accuracy of dental evaluations. Automated detection also ensures uniform results across different cases. This leads to faster and more reliable identification of dental issues.

1.3 Project Objectives

The main objective of this project is to build an efficient dental X-ray classification model capable of identifying various conditions accurately. It aims to automatically detect dental anomalies without the need for continuous manual examination. By automating the process, the system significantly reduces human effort and speeds up diagnostic workflows. This automation helps dentists focus more on treatment rather than image inspection. The model also enhances accuracy by minimizing human error. Overall, it ensures consistent and reliable results across all dental X-ray evaluations..

1.4 Project Scope

The system is versatile and can be effectively used in clinics, hospitals, and academic research settings. It allows users to easily upload dental X-ray images for analysis. Once uploaded, the model performs real-time detection of dental conditions. It also highlights the identified areas through clear annotations. This makes the diagnostic process faster, more interactive, and highly useful for professionals and students alike.

2. Literature Survey

2.1 Existing System Analysis

Traditional diagnosis of dental X-rays relies primarily on the knowledge and experience of dental professionals, making it highly subjective and prone to human error [1]. Manual interpretation becomes challenging when images are unclear or contain subtle abnormalities, which may lead to misdiagnosis or oversight [2]. Factors such as practitioner fatigue, workload, and visual limitations further increase the risk of inaccuracies in clinical evaluation [3]. Additionally, diagnostic outcomes may vary from one practitioner to another, resulting in inconsistent assessments for the same X-ray image [4]. These limitations highlight the need for automated and standardized diagnostic systems that can provide reliable assistance in dental imaging [5].

2.2 Research Papers Review

Research indicates that Convolutional Neural Networks (CNNs) and YOLO (You Only Look Once) models have demonstrated exceptional performance in medical image analysis tasks [6]. CNNs are effective at extracting hierarchical visual features, enabling them to detect fine-grained patterns in modalities such as X-rays, MRI, and CT scans [7]. YOLO, with its real-time detection capability, identifies anomalies rapidly and accurately within a single pass of the image [8]. Studies show that CNN-based systems achieve high sensitivity in detecting lesions and other dental abnormalities, while YOLO models excel at localizing multiple regions of interest simultaneously [9]. These models significantly reduce human error, improve diagnostic accuracy, and accelerate clinical workflows, making them valuable tools in healthcare applications [10].

2.3 Similar Systems Study

Existing medical imaging systems widely leverage deep learning, particularly CNN and YOLO architectures, to analyze complex clinical images with high accuracy [11]. CNN s contribute significantly to classification and segmentation by recognizing patterns such as tumors, fractures, and lesions in dental and general medical imaging [12]. YOLO enhances these capabilities through rapid object detection, allowing clinicians to identify multiple anomalies in real time [13]. Research highlights the complementary nature of CNN and YOLO models, showing that their

integration can reduce diagnostic time and improve the reliability of clinical assessments [14]. These methods have been applied successfully across imaging modalities including X-ray, CT, MRI, and panoramic dental radiography, establishing their importance in automated diagnostic systems [15].

2.4 Technology Review

The proposed system utilizes a combination of advanced deep learning techniques and practical web-based deployment technologies [16]. YOLOv8 provides state-of-the-art object detection performance with improved speed and accuracy compared to previous versions [17]. Python serves as the primary development environment due to its extensive support for machine learning libraries. Open CV is employed for image preprocessing and visualization tasks, enabling effective handling of dental X-ray data [18]. Deep learning frameworks form the core of the detection and classification operations within the system, ensuring robust and automated analysis [19]. A Flask-based web interface facilitates seamless deployment, allowing real-time interaction between end-users and the detection model in clinical or research settings [20].

3. System Analysis

3.1 Requirement Gathering

Requirements gathering for this project involved a detailed collection of all necessary resources and inputs. Medical image datasets were compiled from reliable sources to ensure diverse and representative samples. Each image was carefully annotated to mark areas of interest, such as anomalies or regions requiring classification. Information on user interactions was gathered to design an intuitive interface for uploading and viewing results. Hardware and software requirements were also identified, including GPU support for model training and Python libraries for image processing. Security and privacy considerations were included to protect sensitive medical data. Workflow requirements were defined to streamline image upload, processing, and output display. Overall, this step ensured a clear understanding of all functional and non-functional needs for the system.

3.2 Feasibility Study

The feasibility study of this project highlights its practicality across multiple aspects. Technically, the system is feasible by leveraging YOLOv8 for high-accuracy detection combined with Python for development and integration. Economically, it is cost-effective as it primarily uses free and open-source tools such as Open CV, Flask, and pr-trained models. Operationally, the web-based interface ensures ease of use for clinicians and researchers without requiring specialized software. Implementation can be carried out on standard hardware with GPU support for faster processing. Maintenance is straightforward due to Python's extensive community support. The system can be scaled or updated with minimal effort. Overall, the project demonstrates strong feasibility in technical, economic, and operational terms.

.

3.2.1 Technical Feasibility

The technical feasibility of this project is strong, as it leverages YOLOv8 for accurate and real-time object detection. Python serves as the primary development language, providing extensive libraries for deep learning and image processing. Open CV supports efficient handling and prepossessing of medical images. The integration of YOLOv8 with Python allows seamless model training, testing, and deployment. The system can run on standard hardware with GPU acceleration for faster

computation. Pr-trained models can be utilized to reduce development time and improve accuracy. Overall, the technical approach ensures a reliable and efficient solution for medical image analysis.

3.2.2 Economic Feasibility

The economic feasibility of this project is highly favorable, as it primarily relies on free and open-source tools. Python, Open-CV, and YOLOv8 are all freely available, reducing software licensing costs. Using pr-trained models further minimizes the need for expensive computational resources. Deployment with Flask eliminates the need for costly web development platforms. Hardware requirements are moderate, allowing the system to run on standard machines with GPU support. Overall, this approach ensures a cost-effective solution for medical image analysis.

3.2.3 Operational Feasibility

The operational feasibility of this project is supported by its easy-to-use web-based interface. Users can upload medical images directly through the browser without needing specialized software. The interface displays results clearly, including detected anomalies and annotated outputs. Flask provides a lightweight framework for smooth interaction between the user and the model. The system supports real-time processing, allowing immediate feedback to the user. Minimal training is required, making it accessible to clinicians and researchers. Overall, the web-based design ensures efficient and user-friendly operation.

3.3 SWOT Analysis

Strength:

The system's primary strength lies in its high accuracy in detecting and classifying medical anomalies. Using YOLOv8 and CNN-based models ensures precise identification of regions of interest in medical images. Automated detection reduces human error and improves diagnostic consistency. Real-time processing further enhances efficiency in clinical workflows. Overall, accuracy and reliability are key advantages of this technology.

Weakness:

A notable weakness is the dependency on GPU hardware for efficient training and real-time inference. Systems without GPU support may experience slow

processing times, limiting scalability. Additionally, initial setup and configuration of deep learning environments can be technically challenging for non-experts. This requirement may restrict adoption in smaller clinics with limited resources.

Opportunity:

There is a significant opportunity for implementing this system in hospitals, dental clinics, and research institutions. It can assist clinicians in faster diagnosis, reducing workload and improving patient care. Integration with existing medical software or medicine platforms can further expand its utility. The growing interest in AI-assisted diagnostics makes this project highly relevant.

Threat:

Data privacy and security concerns pose a potential threat. Medical images contain sensitive patient information that must be protected under regulations like HIPAA or GDPR. Any breach or mishandling of data could have legal and ethical implications. Additionally, reliance on accurate labeling and high-quality datasets is crucial; poor data quality could reduce effectiveness.

4. System Requirements Specification

4.1 Functional Requirements

1. Upload X-ray Images:

The system must allow users to upload dental X-ray images in standard formats such as JPG, PNG, or DICOM. Users should be able to select files from their local device and submit them through a web-based interface. The system should validate the uploaded files to ensure they meet format and size requirements. This functionality is essential as it serves as the entry point for image analysis.

2. Detect Dental Diseases:

Once an image is uploaded, the system should automatically process it using the YOLOv8 model and deep learning algorithms. The model will analyze the X-ray to detect various dental anomalies such as cavities, gum disease, or impacted teeth. Detection should be accurate and fast, providing real-time or near real-time results. This requirement ensures that the system can assist clinicians in diagnosis effectively.

3. Display Results:

The system should visually present the detection results on the web interface. Detected anomalies should be highlighted or annotated on the X-ray image, and corresponding labels should be displayed. Additional information, such as the type of disease and confidence score, can be shown to help clinicians interpret results. The results display must be clear, user-friendly, and easy to understand, allowing immediate insights for medical decision-making.

4.2 Non-Functional Requirements

1. Accuracy:

The system must provide precise and correct detection of dental anomalies. High accuracy is critical to ensure clinicians can rely on the results for diagnosis. The model should minimize false positives and false negatives to maintain trust in automated analysis. Continuous evaluation and validation against labeled datasets help achieve and maintain high accuracy.

2. Speed:

The system should process and analyze uploaded X-ray images quickly, ideally in real-time or within a few seconds. Fast response times enhance operational efficiency, allowing clinicians to receive instant feedback. Efficient processing ensures the system is practical for use in busy clinical environments.

3. Reliability:

The system must operate consistently without failures or crashes. It should handle multiple users or image uploads simultaneously without performance degradation. Regular testing, robust coding practices, and error handling mechanisms contribute to overall system reliability.

4. Usability:

The web interface should be intuitive, easy to navigate, and require minimal training. Users should be able to upload images, view results, and understand outputs effortlessly. Clear annotations, labels, and instructions improve the user experience, making the system suitable for both technical and non-technical users.

4.3 Hardware Requirements

1. RAM(8Gbormore):

The system requires at least 8GB of RAM to efficiently load and process medical images. Sufficient memory ensures smooth execution of deep learning models, image pre processing, and web application tasks without slowing down the system. More RAM may be needed for handling larger datasets or batch processing multiple images.

2. GPU(Optional but Recommended):

While the system can run on CPU, having a GPU significantly accelerates model training and inference. A compatible GPU (e.g., NVIDIA with CUDA support) allows YOLOv8 and other deep learning models to process images faster, enabling near real-time detection. GPU usage is particularly important for large-scale deployments or high-resolution images.

3. Processor(Inteli5orhigher):

A modern processor, such as Intel i5 or above, ensures the system runs efficiently. The CPU handles tasks like image prepossessing, running the Flask web

server, and managing concurrent user requests. Higher processing power improves the responsiveness and overall performance of the application.

4. Storage:

Adequate storage is required to save the operating system, software libraries, models, uploaded images, and results. SSD s are recommended for faster data access and reduced latency during image processing.Overall, this hardware setup ensures the system operates efficiently, with optional GPU support providing enhanced speed for deep learning tasks.

4.4 Software Requirements

1. Python:

Python is the primary programming language used for building the detection system. It provides powerful libraries for machine learning, image processing, and web development. Its simplicity and extensive community support make it ideal for rapid development and deployment.

2. YOLOv8:

YOLOv8 (You Only Look Once version 8) is the core deep learning model used for detecting dental diseases. It offers high accuracy and fast object detection, making it suitable for real-time medical image analysis. YOLOv8 supports both CPU and GPU execution, giving flexibility during training and testing.

3. Flask:

Flask is a lightweight Python web framework used to create the application interface. It manages user interactions such as uploading X-ray images and displaying detection results. Flask allows easy integration of back-end deep learning models with a clean, user-friendly fronted.

4. Open CV:

Open CV is an essential library for image processing tasks, including re sizing, cropping, filtering, and prepossessing X-ray images before sending them to the model. It enhances image quality and prepares the data for accurate detection.

5. Google Colab:

Google Colab is used for training and testing the YOLOv8 model in the cloud. It provides free GPU access, eliminating the need for expensive local hardware. Colab also supports seamless integration of Python, YOLO, and dataset handling.

If you want, I can also provide a software requirements table or a combined software + hardware requirements section for your project documentation.

4.5 Network Requirements

1. Stable Internet Connection:

A reliable and stable internet connection is essential, especially when using cloud-based platforms like Google Colab for training the YOLOv8 model. A stable connection prevents session interruptions and ensures smooth execution of training code and dataset uploads.

2. Sufficient Upload Speed:

Since medical image datasets can be large, good upload speed is needed to transfer images to Colab or any cloud storage. Faster upload speeds reduce delays during dataset preparation and model training.

3. Adequate Download Speed:

Downloading pre-trained models, libraries, or results from the cloud requires reasonable download speed. This ensures quick setup and efficient retrieval of trained model weights or output files.

4. Browser-Based Access:

Because the system uses a web interface (Flask) and cloud platforms like Colab, users must have internet access to interact with the model, upload images, and view processed outputs.

5. Low Latency for Real-Time Tasks:

If the model is deployed on a remote server, low network latency helps maintain fast response times during inference. This ensures smooth user experience when uploading X-rays and receiving results.

5. System Design

5.1 Architecture Design

- The User uploads a dental X-ray image.
- The Preprocessing module cleans and prepares the image.
- The YOLOv8 model performs detection on teeth or anomalies.
- The Result Generation step prepares annotated output.
- The Result Display shows the detection visually on a web page.

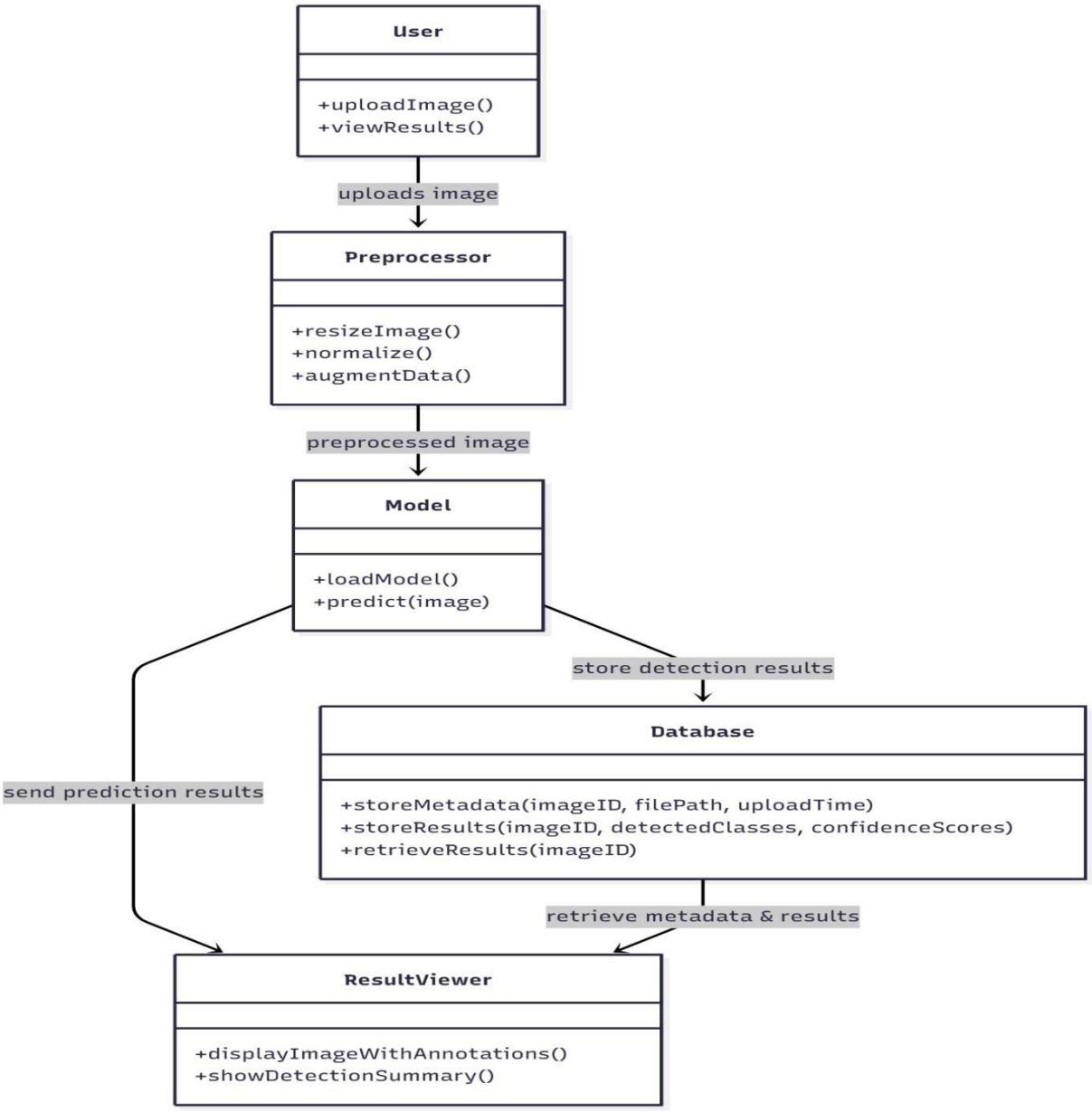

Fig 1: Architecture Design

5.2UML Diagrams:

5.2.1 Use Case Diagrams

The Use-Case Diagram shows how the user interacts with the Dental X-ray Detection System.It identifies the main actor, User, and the system-level functions available.The two primary use cases are Uploading Dental X-ray Images and Viewing Detection Results.These use cases represent what the system must provide to the end user.The diagram focuses on external behavior rather than internal processing.
Overall, it helps understand user goals and system boundaries clearly.

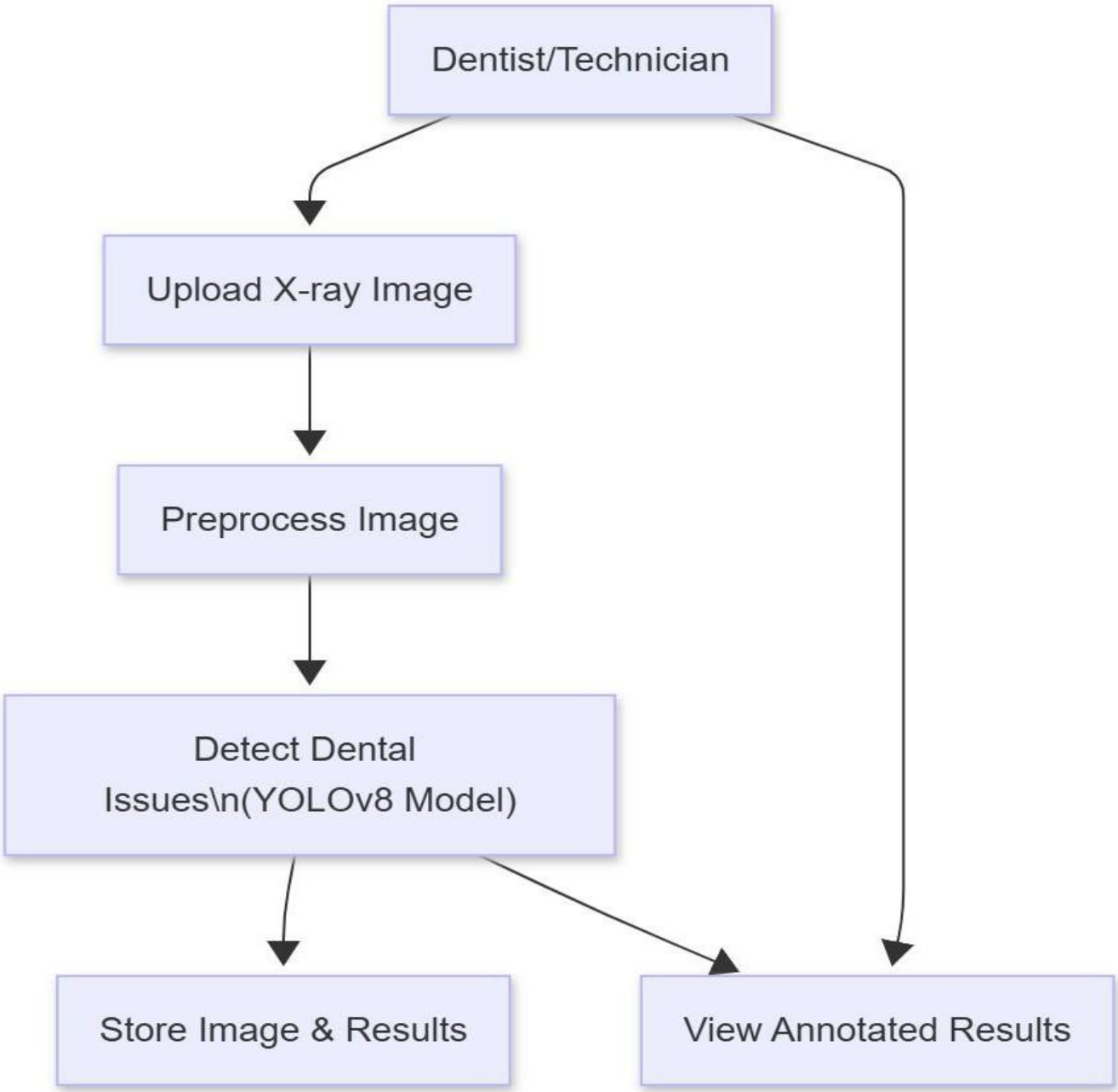

Fig 2: Use Case Diagrams

5.2.2 Class Diagram:

The Class Diagram represents the structural design of the system.It includes major classes such as. User, Preprocessing, YOLOv8 Model, and Result.Each class contains specific attributes and methods related to its functionality The relationships show how components interact, such as preprocessing feeding data to the model.It defines the blueprint of the system's architecture in an object-oriented format.This helps developers understand how the system is organized internally.

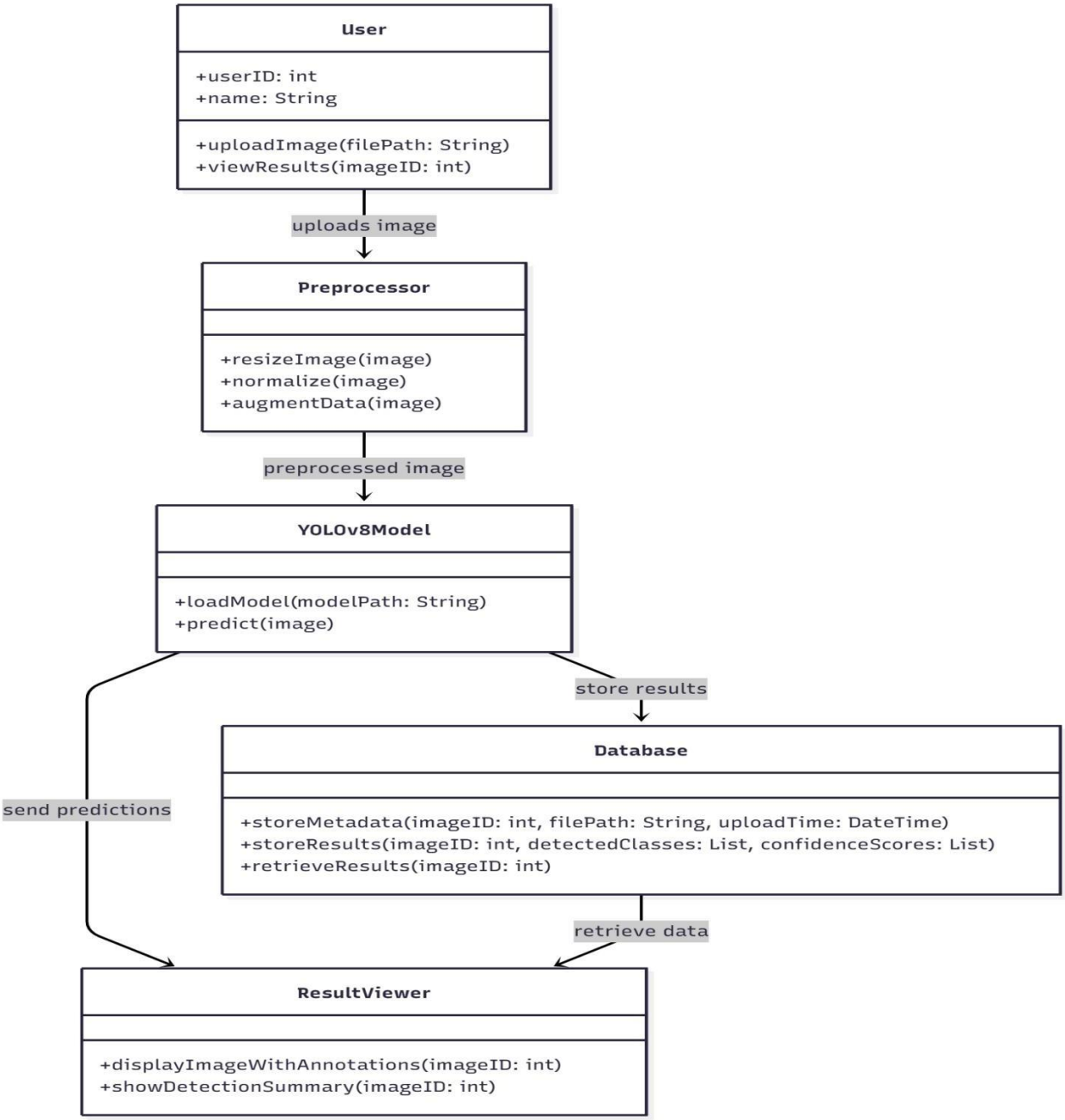

Fig 3: Class Diagram

5.2.3 Sequence Diagrams:

The Sequence Diagram describes the step-by-step flow of operations in the system.It starts with the user uploading an image, triggering preprocessing and model detection. Each interaction is shown as a message passed between system components.

I thigh lights order,timing,and dependencies between actions. The diagram ensures that all components work together in the correct sequence.It is useful for understanding the runtime behavior of the system.

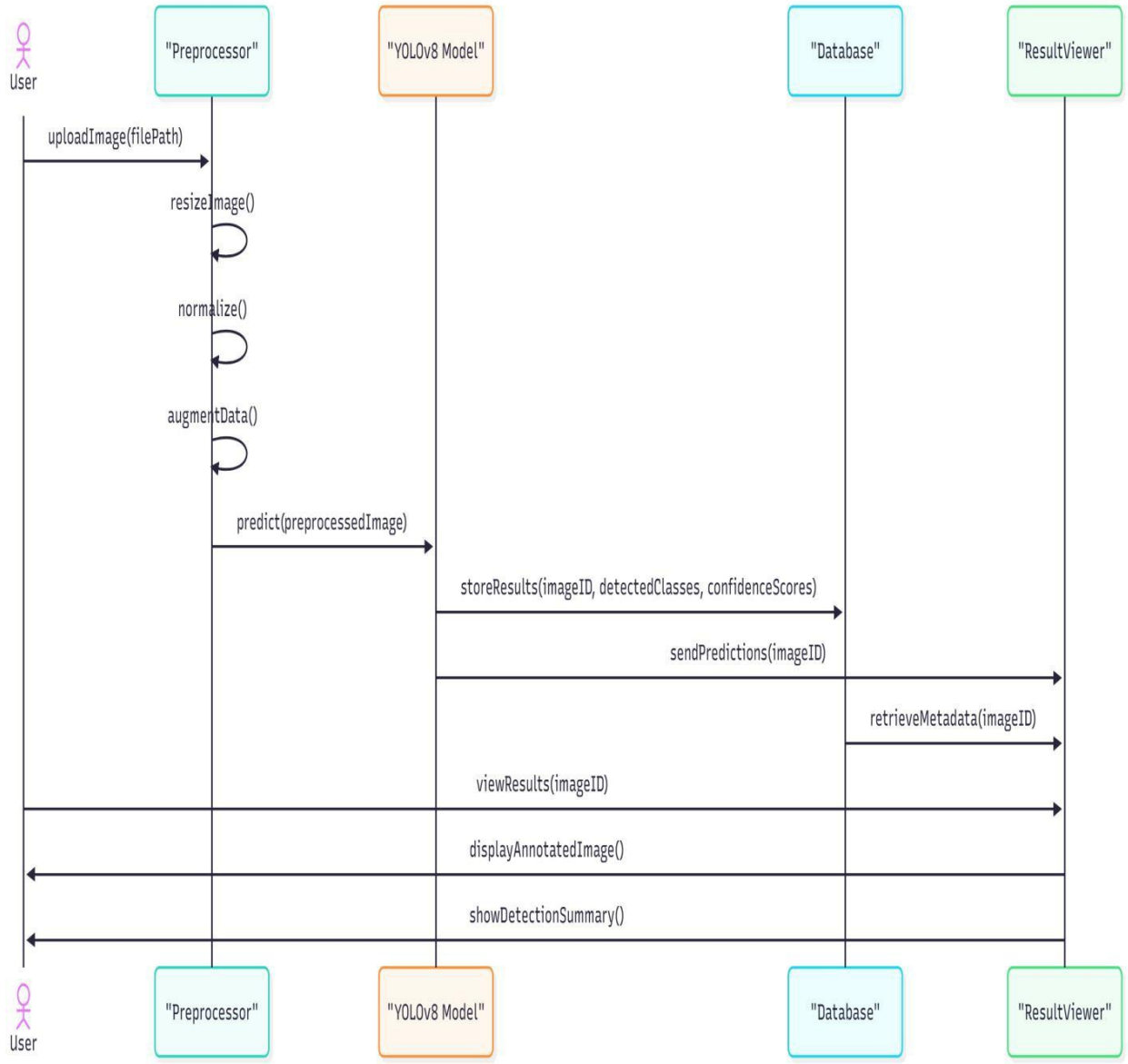

Fig 4: Sequence Diagrams

5.2.4 Activity Diagrams:

The Activity Diagram illustrates the workflow of the dental X-ray detection process.It begins with the user uploading an image and ends with the system displaying results.Actions, decisions, and transitions show how the system progresses step by step.The diagram includes branching logic for detection operations. It visualizes the dynamic behavior and process logic of the system.Overall, it helps map the entire operational flow clearly and logically.

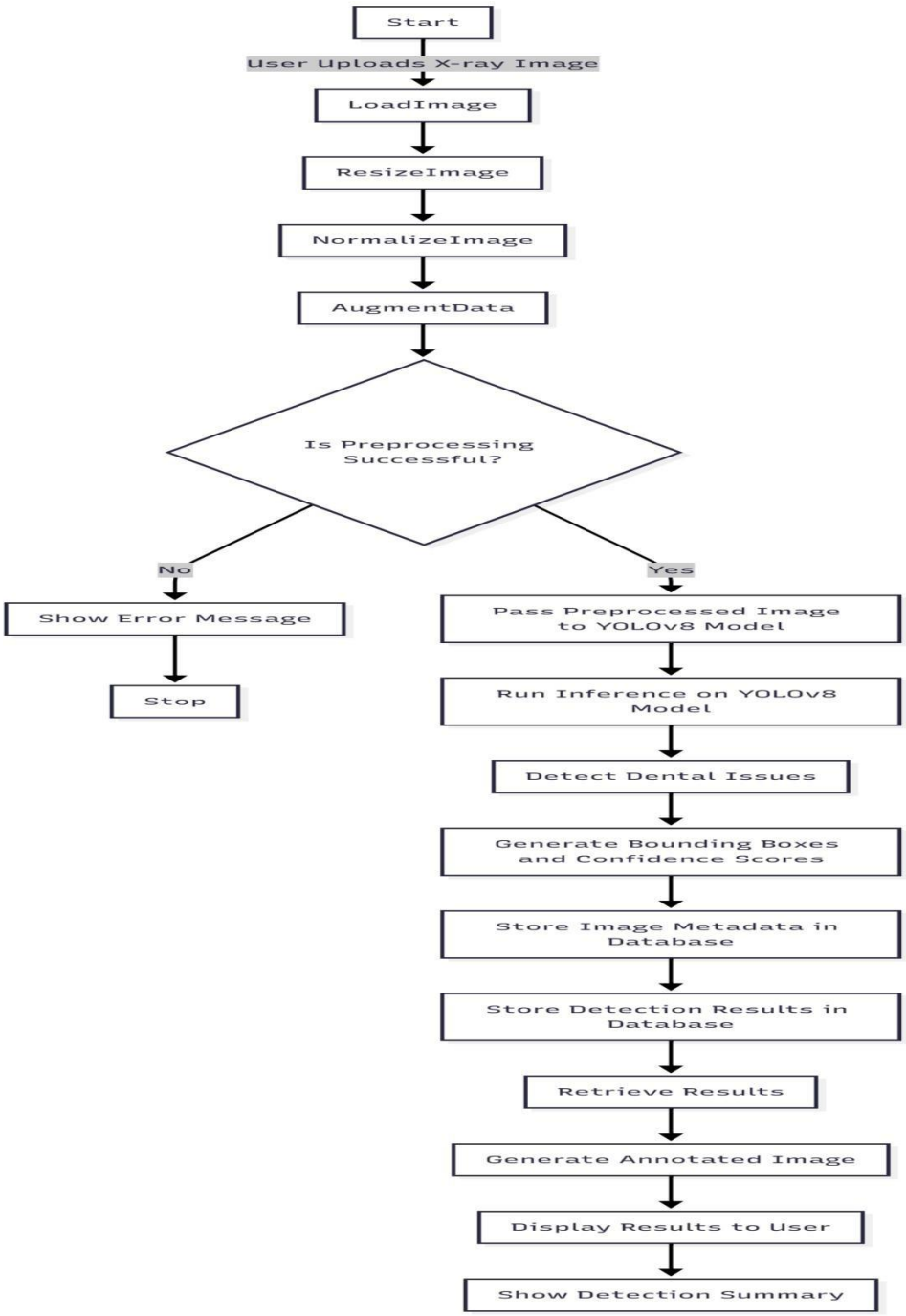

Fig 5: Activity Diagrams

5.3 Database Design:

5.3.1 ER Diagrams:

The database design stores only image metadata, while the physical image files are kept in local folders for efficient access. Each uploaded dental X-ray is assigned a unique ID, and its file name, file path, and upload time are recorded. After processing by the YOLOv8 model, the database logs the output image path and detection results such as detected classes and confidence scores. Storing metadata instead of full image binary data keeps the database lightweight. This structure also allows fast retrieval of images during result display. Such a design is simple, scalable, and ideal for local ML-based applications.

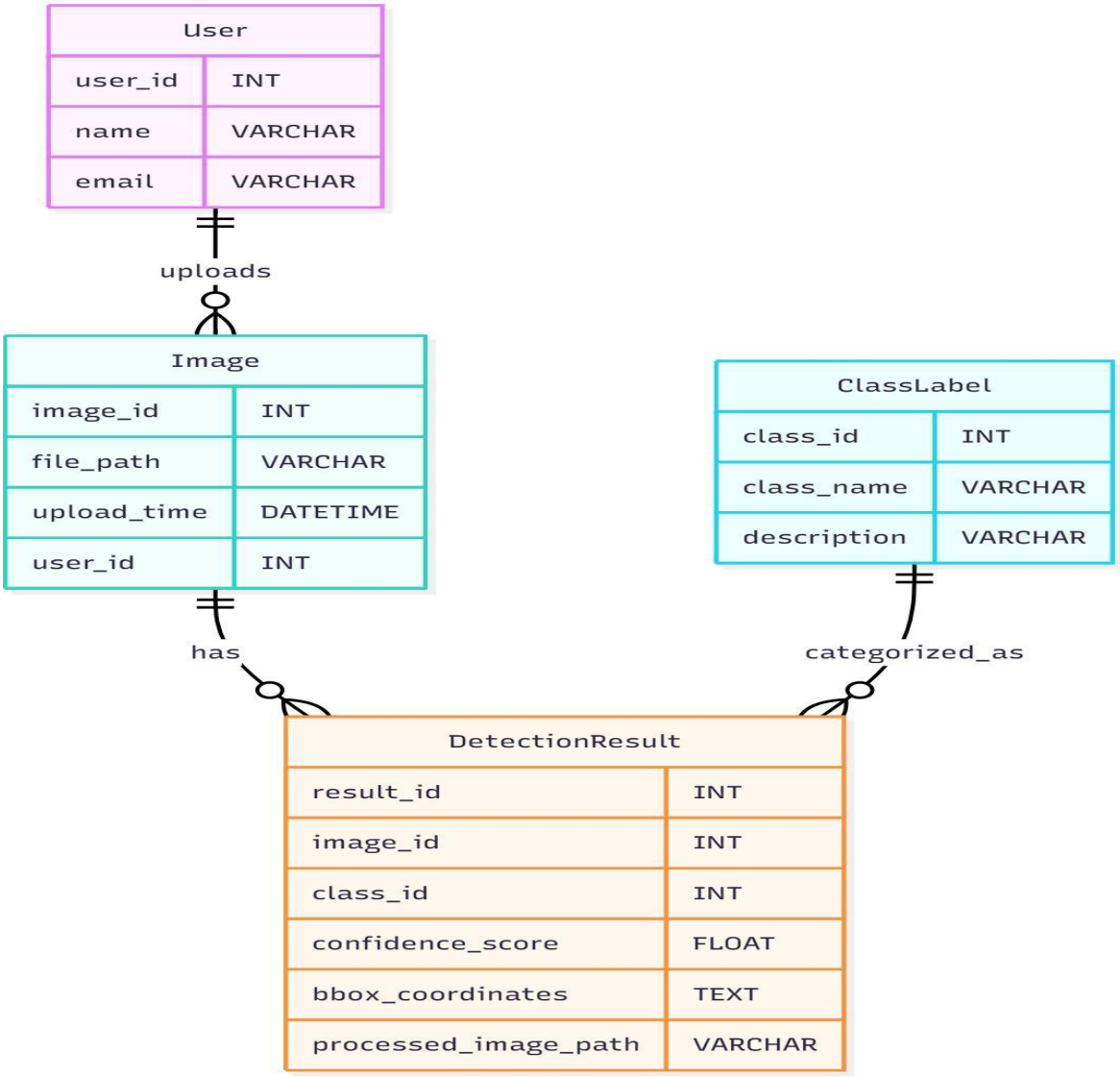

Fig 6: ER Diagrams

5.3.2 Data Dictionary:

In a dental X-ray image classification system using machine learning, the data dictionary provides a structured overview of all the features used for training, analysis, and prediction. These attributes help in organizing image information, improving preprocessing accuracy, and ensuring proper interpretation of model results. Common fields include Image ID, a unique identifier for each X-ray; File Path, indicating the stored location of the image; Image Size/Resolution, specifying the dimensions of the X-ray; and Patient Age or Tooth Region (if available), which provide additional diagnostic context. The target variable, typically Dental Condition, represents categories such as cavity, fracture, root infection, or healthy tooth. This data dictionary ensures clear feature definitions, supporting effective preprocessing, model training, and explanation of classification outcomes.

Table1 – Data Dictionary

Attribute Name	**Description**	**Data Type**	**Example/Value**
Image_ID	Unique identifier for each dental X-ray image	String	IMG_00123
File_Path	Location of the stored X-ray image	String	/images/xray_00123.png
Image_Format	File type of the image	String	PNG, JPG
Resolution	Dimensions of the image in pixels	String	1024 × 1024
Upload_Date	Date the image was uploaded to the system	Date	2025-02-14
Patient_Age (optional)	Age of the patient related to the X-ray	Integer	32
Tooth_Region (optional)	Region of interest	String	Molar
Preprocessed_Flag	Indicates whether the image has been	Boolean	Boolean True / False

Attribute Name	Description	Data Type	Example/Value
Augmentation_Type	Type of augmentation applied during training	String	Rotate, Flip, Brightness
Model_Prediction	Predicted dental condition	String	Cavity
Confidence_Score	Probability/score of the prediction	Float	0.92
Ground_Truth_Label	Actual diagnosis from dataset	String	Cavity
Bounding_Box_Data (if using object detection)	Coordinates of detected regions (if applicable)	JSON	{x1, y1, x2, y2}
Processing_Time	Time taken by model to classify the image	Float	0.45 sec

6. Implementation

6.1 Algorithms

During training:

1. Input image → standard preprocessing + augmentations (mosaic, mixup, HSV jitter, flips, scaling).
2. Forward pass through backbone → neck → decoupled head. Head outputs per-cell: objectness score, class logits, and box representation (distributional regression bins used by DFL).
3. Label assignment (anchor-free) — ground truth boxes are assigned to locations (centers / grid cells) instead of anchor matching.
4. Compute losses: classification loss (BCE / focal variants), box regression loss (CIoU / Complete-IoU) plus Distribution Focal Loss (DFL) for box distribution refinement. Total loss = cls + box + dfl.
5. Backprop & optimizer step (Ultralytics supports Adam/AdamW/SGD options and automatic selection/one-cycle schedules in training scripts).

During inference:

1. Single forward pass produces many candidate boxes (objectness + class scores + box coords expressed via predicted distributions).
2. Decode distributional outputs to bounding boxes (DFL → continuous box coordinates).
3. Score thresholding → Non-Maximum Suppression (NMS) to remove overlaps (standard NMS or soft-NMS variants).
4. Final boxes returned.

Key loss / training details (why they matter)

- DFL (Distribution Focal Loss) improves regression by modeling box edges as a discrete distribution then converting to continuous coordinates — helps precise localization.
- CIoU (Complete-IoU) for box loss accounts for overlap, center distance and aspect ratio — stabilizes box learning.
- Decoupled head lets classification and regression learn independently, often improving mAP vs a coupled head.

6.2 Pseudo code

```
# Pseudocode for Dental X-ray Image Classification

# Step 1: Import necessary libraries

    - pandas, numpy

    - cv2 (for image processing)

    - os (for directory navigation)

    - train_test_split from sklearn.model_selection

    - ML model (

    - accuracy_score, confusion_matrix, classification_report from sklearn.metrics


# Step 2: Load and prepare the image dataset

Set dataset_path = "dental_xray_dataset/"

Initialize empty lists:

    images = []

    labels = []

For each folder in dataset_path:

        For each image in folder:

                Read the image using cv2

                Convert image to grayscale

                Resize image to fixed size (e.g., 128×128)

                Normalize pixel values by dividing by 255

                Flatten the image into 1D array

                Append flattened image to images list

                Extract folder name as class label

                Append label to labels list
```

```
convert images  and labels  to numpy arrays

# Step 3: Prepare Feature Matrix and Labels

X = images            # Feature matrix

y = labels           # Target labels

# Step 4: Split the dataset

Split X and y into:
        X_train, X_test
        y_train, y_test
Use train_test_split(test_size=0.3, random_state=42)

# Step 5: Initialize Machine Learning Model
Choose ML model (example: SVM)
model = SVM(kernel='rbf', C=1.0)

# Step 6: Train the model
model.fit(X_train, y_train)

# Step7:Perform Predictions
y_pred = model.predict(X_test)

# Step 8: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

# Step 9: Display results   Print("Accuracy:",
accuracy) Print("Confusion Matrix:",
conf_matrix) Print("Classification Report:",
class_report)
```

#Step10: Save the model

Save model using pickle or joblib:

joblib.dump(model, "dental_xray_ml_model.pkl")

6.3 Source code

```
import os
from pathlib import Path
from flask import Flask, render_template, request, redirect, url_for, flash, abort
from werkzeug.utils import secure_filename
from werkzeug.security import generate_password_hash, check_password_hash
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user, current_user
from ultralytics import YOLO
import shutil
from functools import wraps

app = Flask(__name__)
app.secret_key = 'your_secret_key'  # Change this to a random secret key

# Database configuration
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)
login_manager = LoginManager(app)
login_manager.login_view = 'login'

# Folder setup
UPLOAD_FOLDER = "uploads"
OUTPUT_FOLDER = "static/outputs"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(OUTPUT_FOLDER, exist_ok=True)
```

```
# Model path
MODEL_PATH = "model/dental_model.pt"

# Load YOLO model
try:
    model = YOLO(MODEL_PATH)
    print("Model loaded successfully from:", MODEL_PATH)
except Exception as e:
    print(f" Failed to load YOLO model: {e}")
    model = None

# User model
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(150), unique=True, nullable=False)
    password_hash = db.Column(db.String(150), nullable=False)
    is_admin = db.Column(db.Boolean, default=False)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))

# Ensure DB tables exist
with app.app_context():
    db.create_all()
```

```
def admin_required(func):
    @wraps(func)
    def decorated_view(*args, **kwargs):
        if not current_user.is_authenticated or not current_user.is_admin:
            abort(403)
        return func(*args, **kwargs)
    return decorated_view

# ---------------------------------------------
#     HOME / UPLOAD + PREDICT
# ---------------------------------------------
from flask import current_app

@app.route("/", methods=["GET", "POST"])
def root_redirect():
    return redirect(url_for("login"))

@login_required
@app.route("/home", methods=["GET", "POST"])
def upload_predict():
    if request.method == "POST":
        if "file" not in request.files:
            return render_template("index.html", error="No file uploaded!")

        file = request.files["file"]
        if file.filename == "":

            return render_template("index.html", error="No file selected!")

        filename = secure_filename(file.filename)
        filepath = os.path.join(UPLOAD_FOLDER, filename)
        file.save(filepath)
```

```
        if model is None:
            return render_template("index.html", error="Model not loaded properly!")

        # Run YOLO inference
        results = model.predict(source=filepath, save=True, imgsz=640, conf=0.5)

        save_dir = Path(results[0].save_dir)
        img_name = Path(results[0].path).name
        output_path = save_dir / img_name

        static_output_path = Path(OUTPUT_FOLDER) / img_name
        shutil.copy(output_path, static_output_path)

        detections = []
        for box in results[0].boxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            label = model.names[cls_id]
            detections.append(f"{label} ({conf:.2f})")

        return
            render_template( "index.
            html",
            uploaded_image=url_for("static", filename=f"../{filepath}"),
            output_image=url_for("static", filename=f"outputs/{img_name}"),
            prediction=", ".join(detections) if detections else "No detections found."
        )

    return render_template("index.html")

# ------------------------------------------
#     LOGIN ROUTE
# ------------------------------------------
@app.route("/login", methods=["GET", "POST"])
```

```
def login():
    if current_user.is_authenticated:
        return redirect(url_for("upload_predict"))
    if request.method == "POST":
        email = request.form.get("email")
        password = request.form.get("password")

        user = User.query.filter_by(email=email).first()
        if user and user.check_password(password):
            login_user(user)
            return redirect(url_for("upload_predict"))
        else:
            flash("Invalid credentials!", "error")
            return render_template("login.html")
    return render_template("login.html")

# ------------------------------------------
#     REGISTER ROUTE
# ------------------------------------------
@app.route("/register", methods=["GET", "POST"])
def register():
    if current_user.is_authenticated:
        return redirect(url_for("upload_predict"))
    if request.method == "POST":
        email = request.form.get("email")
        password = request.form.get("password")
        existing_user = User.query.filter_by(email=email).first()
        if existing_user:
            flash("Email already exists!", "error")
            return render_template("register.html")
        if not email or not password:
            flash("Email and password are required!", "error")
            return render_template("register.html")
```

```
        new_user = User(email=email)
        new_user.set_password(password)
        # First registered user is admin by default
        if User.query.count() == 0:
            new_user.is_admin = True
        db.session.add(new_user)
        db.session.commit()
        flash("Registration successful! Please login.", "success")
        return redirect(url_for("login"))
    return render_template("register.html")

# ---------------------------------------------
#     LOGOUT ROUTE
# ---------------------------------------------
@app.route("/logout")
@login_required
def logout():
    logout_user()
    flash("You have been logged out.", "success")
    return redirect(url_for("login"))

# ---------------------------------------------
#     HISTORY PAGE
# ---------------------------------------------
@app.route("/history")
@login_required
def history():
    # Dummy history data example (can be extended for real user activity)
    history_data = [
        {"date": "2023-10-01", "action": "Uploaded dental X-ray #1"},
        {"date": "2023-10-05", "action": "Uploaded dental X-ray #2"},
        {"date": "2023-10-10", "action": "Viewed analysis results"},
    ]
```

```
    return render_template("history.html", history=history_data)

# ---------------------------------------------
#     ADMIN PAGE
# ---------------------------------------------
@app.route("/admin", methods=["GET", "POST"])
@login_required
@admin_required
def admin():
    if request.method == "POST":
        action = request.form.get("action")
        if action == "add":
            email = request.form.get("email")
            password = request.form.get("password")
            is_admin_flag = request.form.get("is_admin") == "on"
            if email and password:
                user = User.query.filter_by(email=email).first()
                if user:
                    flash("User already exists.", "error")
                else:
                    new_user = User(email=email, is_admin=is_admin_flag)
                    new_user.set_password(password)
                    db.session.add(new_user)
                    db.session.commit()
                    flash("User added successfully.", "success")
            else:
                flash("Email and password are required to add user.", "error")
        elif action == "edit":
            user_id = request.form.get("user_id")
            user = db.session.get(User, user_id)
            if user:
                email = request.form.get("email")
                password = request.form.get("password")
```

```
            is_admin_flag = request.form.get("is_admin") == "on"
            if email:
                user.email = email
            if password:
                user.set_password(password)
            user.is_admin = is_admin_flag
            db.session.commit()
            flash("User updated successfully.", "success")
        else:
            flash("User not found.", "error")
    elif action == "delete":
        user_id = request.form.get("delete_user_id")
        user = db.session.get(User, user_id)
        if user:
            db.session.delete(user)
            db.session.commit()
            flash("User deleted successfully.", "success")
        else:
            flash("User not found.", "error")

    users = User.query.all()
    return render_template("admin.html", users=users)

# ----------------------------------------
# RUN FLASK APP
# ----------------------------------------
if __name___== "__main__":
    app.run(debug=True)
```

6.4 Database Queries

```
import mysql.connector

# Function to connect to the MySQL database
def project_db():
    try:
        # Establish the database connection
        connection = mysql.connector.connect(
            host="localhost",      # Host where MySQL server is running
            user="root",           # Default MySQL username
            password="",           # Your MySQL password (empty in XAMPP/WAMP)
            database="dental_xray_classifier"  # Your database name
        )

        if connection.is_connected():
            print("Database connection successful!")
            return connection

    except mysql.connector.Error as err:
        print(f"Error: {err}")
        return None


# Test the database connection
if __name__ == "__main__":
    db = project_db()
    if db:
        print("Closing the database connection.")
        db.close()
```

7. Testing

7.1 Test Plan

1. Objective:

To verify that the dental X-ray image classification system accurately classifies images into categories such as cavities, fractures, root issues, or healthy teeth.

2. Test Scope:

Data Integrity: Ensure that the X-ray dataset is complete, correctly labeled, and properly preprocessed Model Accuracy: Test the model's ability to correctly classify dental conditions.

Performance: Assess model processing time, particularly for large batches of images.

Edge Cases: Handle unusual X-ray images, such as images with low quality, partial teeth, or artifacts.

Deployment: Verify that the model functions correctly when integrated into the production system or web interface.

3. Test Types

1.Unit Tests:

- Test individual image preprocessing steps (e.g., grayscale conversion, resizing, noise reduction).
- Validate feature extraction components (e.g., histogram extraction, edge detection, pixel normalization).
- Check model loading, prediction function, and image input validation.

2.Integration Tests:

- Image preprocessing pipeline
- File upload system
- Database (if storing results or metadata)
- APIs or backend services

- Verify that processed images are correctly passed into the model and results returned without errors.

3. System Tests:

- Perform end-to-end testing:
- image upload → prepossessing → model prediction → output result (Healthy / Disease).
- Test the system's ability to handle different image formats (JPG, PNG, JPEG, DICOM).
- Verify UI/UX flow: user login, upload, result display, and error handling.

4. Regression Tests:

- Run the model on the updated dateset whenever new X-ray images are added.
- Ensure that model improvements, parameter tuning, or retraining do not reduce previous accuracy.
- Compare predictions before and after updates to maintain performance stability.

5. Performance Tests:

- Measure the time taken for:
- Uploading an X-ray image
- Prepossessing the image
- Generating a prediction

Test system scalability:

- Processing large batches of images
- Handling concurrent uploads in clinical/hospital environments.

6. Security Tests:

- Ensure uploaded X-ray images are securely stored and not leaked.
- Protect against malicious image uploads (e.g., files disguised as images).
- Test for adversarial attacks that could manipulate model predictions.
- Verify user authentication, session handling, and encrypted communication.

7.2 Test Cases

- Unit Tests:

Test individual preprocessing steps (e.g., re-sizing, normalization, noise reduction).Validate the functionality of each component in the classification pipeline.

- Integration Tests:

Ensure that the classification model integrates correctly with the image upload module, prepossessing pipeline, and result display system.

- System Tests:

Test the end-to-end process from image upload to prediction output (detected dental condition).Verify the system can handle different image formats (.png, .jpg, .jpeg) and resolutions.

- Regression Tests:

Test the model with updated datasets to ensure improvements or retraining do not negatively affect accuracy.

- Performance Tests:

Measure inference time for single and batch image processing.Test scalability for large datasets and multiple concurrent users.

- Security Tests:

Ensure uploaded X-ray images and patient data are protected.Test for vulnerabilities that could compromise system integrity or access to medical images.

4. Test Data

Training Data: Use a dateset of dental X-ray images labeled with dental conditions.

Testing Data: A separate set of images for evaluating model performance.

Edge Case Data: Low-quality, partial, or unusual X-ray images.

Simulated Data: Create augmented images (rotated, brightness-adjusted, occluded) to simulate rare cases not present in the training set.

7.2.1 Unit Testing

Data Preprocessing Functions:

Test image loading functions to ensure images are read correctly.Test image resizing and normalization to confirm the model receives properly scaled inputs.

Test augmentation methods (rotation, flipping) to verify they modify images correctly.

Model Training & Prediction:

Test YOLOv8 training function to ensure the model trains without runtime

errors.Test prediction function to ensure bounding boxes, classes, and confidence scores are generated correctly.

Evaluation Metrics Calculation :

metric computation functions for mAP, precision, recall, and F1-score.

Unit Test Table (Customized for Dental X-ray Image Classification Project)

Table2 – Unit Test

Test ID	**Component Test**	**Description**	**ExpectedOutcome**
UT01	Data Preprocessing	Test image loading and missing image handling	Missing or corruptedimagesshould be detectedandhandledproperly
UT02	Data Preprocessing	Test image resizing & normalization	Images should be resizedtomodelinput size (e.g., 640×640) andnormalized correctly
UT03	Data Preprocessing	Test augmentation functions (flip, rotate, brightness changes)	Augmented images shouldbegenerated without distortionorerrors
UT04	Model Training	Test YOLOv8 model.train() method	Model should begintrainingwithno crashes, logs generatedcorrectly
UT05	Model Prediction	Test YOLOv8 model.predict() on sample X-ray images	Model should output boundingboxes, class labels (cavity, tooth,filling), and confidencescores
UT06	Evaluation Metrics	Test mAP computation	mAP value shouldbecalculatedcorrectly for validationdataset
UT07	Evaluation Metrics	Test precision, recall, F1-score	Metrics should matchexpectedvalues based ontrue/falsepositives

7.2.2 Integration Testing

Data Pipeline Integration:

Ensure image preprocessing steps (loading, resizing, normalization, augmentation) integrate smoothly with the YOLOv8 training and prediction modules. Verify that preprocessed images are correctly passed to the model without shape or format errors.

Model Integration:

Ensure YOLOv8 predictions (detected teeth, cavities, confidence scores) are correctly transmitted to external components such as:

- UI display module
- Result visualization module
- Report generation module

Database Integration:

Verify that prediction results (image ID, output file path, detected classes, confidence scores) are correctly saved to the database.

Integration Test Table (Customized for Dental X-ray Image Classification Project)

Table3 – Integration Test

Test ID	**Component**	**Test Description**	**Expected Outcome**
IT01	Data Pipeline	Test integration of preprocessing with YOLOv8 model training & pass preprocessed images into prediction	Preprocessed images should pass correctly into YOLOv8 prediction without errors
IT02	Model Integration	Test YOLOv8 prediction integration with external modules (e.g., result viewer, UI, API)	Predictions (bounding boxes, class labels, confidence) should be returned and displayed without errors
IT03	Error Handling	Test how system handles corrupted images, missing files, or invalid model outputs	System should log errors and continue running without crashing
IT04	Database Integration	Verify saving of prediction results (image path, output path, detected objects, confidence scores) to database	All prediction results should be stored correctly and retrievable

7.2.3 System Testing

End-to-End Workflow:

The complete workflow—from uploading a dental X-ray image to obtaining the YOLOv8 classification output—should function correctly.he system should correctly detect dental structures (teeth), cavities, fillings, and other abnormalities.Edge cases such as low-quality images, rotated X-rays, over/under-exposed images, or partially visible teeth should be handled properly.

Functionality:

The system should accurately classify and localize dental findings.Predictions should match the ground truth labels in the test dataset, and performance metrics (accuracy, precision, recall, mAP) should be acceptable (e.g., mAP > 90%, precision & recall balanced).

Performance and Scalability:

The system should process each dental X-ray image within an acceptable time frame (e.g., <1–2 seconds depending on GPU).

Table4 – System Testing

Test ID	**Component**	**Test Description**	**Expected Outcome**
ST01	End-to-End Workflow	Test complete system from image upload to YOLOv8 prediction output	System should process the image correctly and display bounding boxes, class labels, and confidence scores
ST02	Functionality	Test classification accuracy using normal and edge-case X- rays (blurred, rotated, low-light images)	System should detect dental issues correctly and match test dataset labels
ST03	Performance & Test Scalability	Test system's processing time under heavy load (multiple images at once)	System should process X-ray images quickly without delays or crashes
ST04	Security	Test secure handling of patient X-ray images and metadata	Images and results should be stored securely with proper access control

7.3 Test Results

Unit Tests should confirm that individual components (image preprocessing, YOLOv8 model training, prediction functions, evaluation metrics) work correctly.Integration Tests should verify that all modules—preprocessing, model, UI, database—interact smoothly.System Tests should confirm that the complete dental X-ray classification system is functional, performs under load, is secure, and handles errors gracefully.

8 Implementation Results

8.3 Screenshots:

Register page: A register is a small, fast storage location within a CPU used to hold temporary data and instructions during processing.

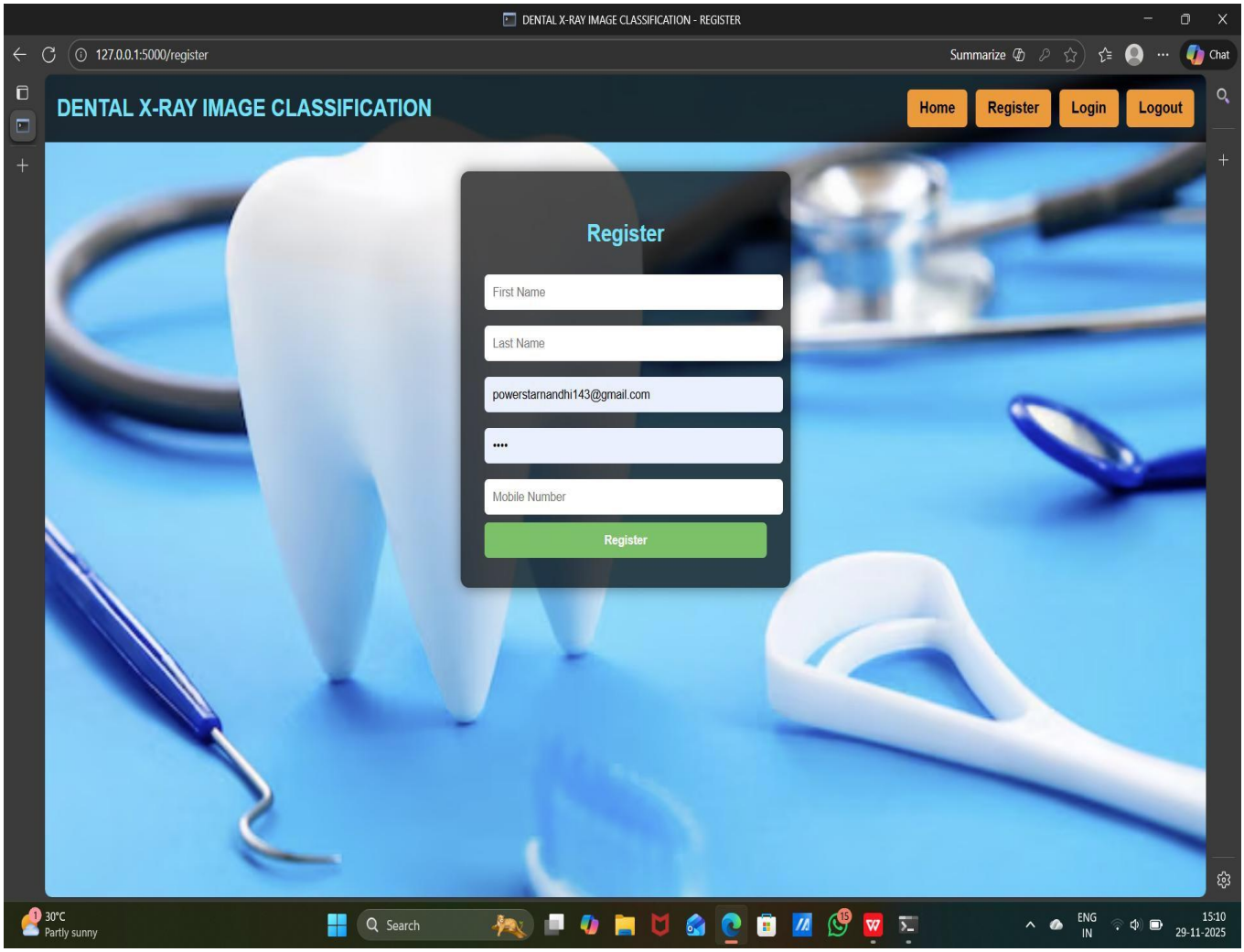

Fig 7: Register Page

The page displays a clean and modern Register interface for the Dental X-Ray Image Classification system. At the center, a dark translucent form card appears with fields for first name, last name, email, password, and mobile number. The background features a high-quality dental-themed image with tools and a large tooth, emphasizing the medical context. The navigation bar at the top includes options like Home, Register, Login, and Logout with bright orange buttons. Overall, the page presents a professional, user-friendly layout designed for easy user registration.

Login page: A login is the process of gaining authorized access to a system or application by providing credentials, typically a username and password.

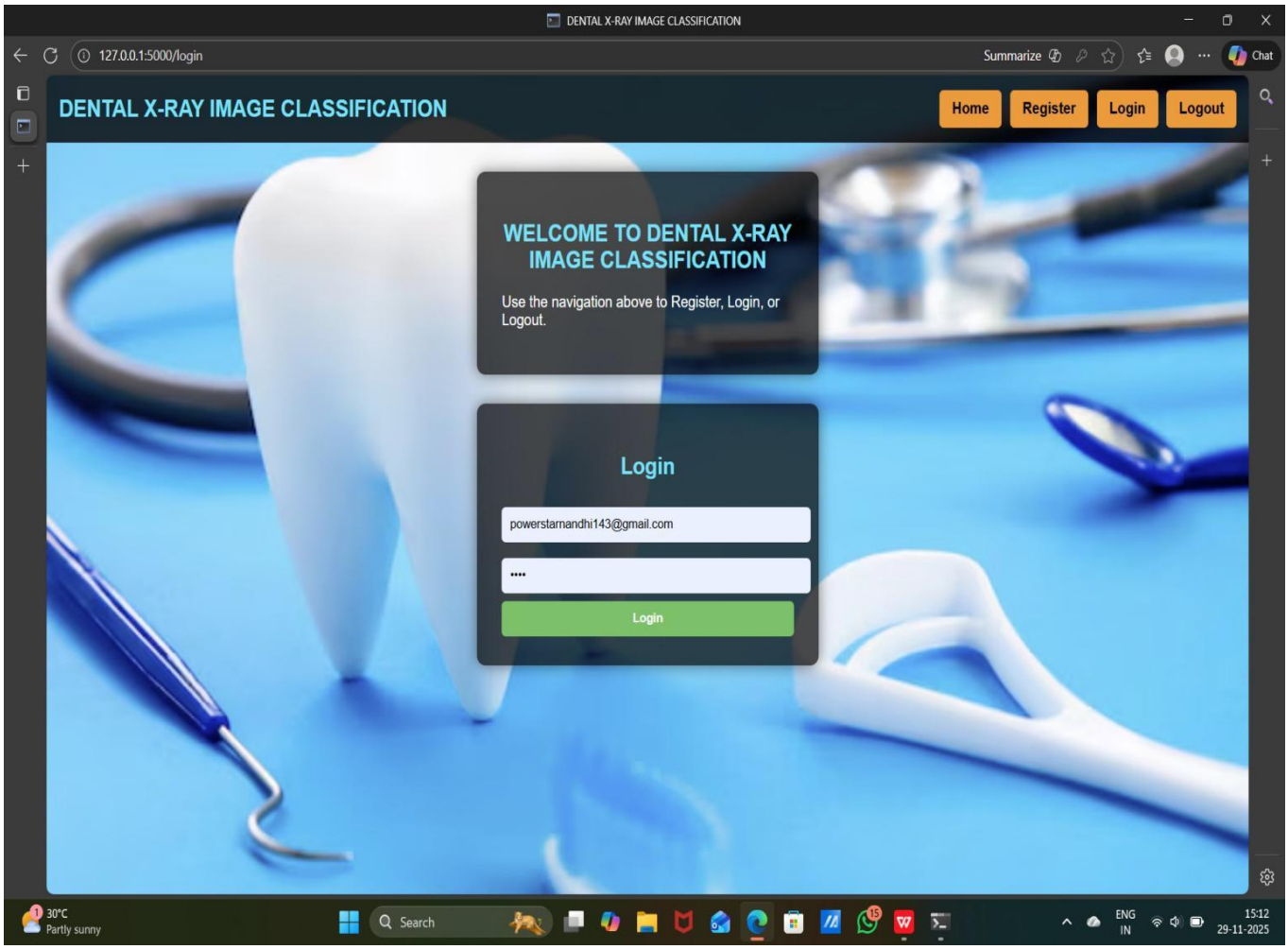

Fig 8: Login Page

The page presents a clean and professional Login interface for the Dental X-Ray Image Classification system. A welcoming message at the top introduces the platform and guides users to use the navigation bar for registration, login, or logout. Below it, a sleek login form is displayed with fields for email and password, placed inside a dark, transparent card. The background features a high-quality dental-themed image with tools and a large tooth, enhancing the medical aesthetic. The design is user-friendly, visually appealing, and structured to ensure easy access to the system.

8.4 Output Screens: The output screens of the Dental X-Ray Image Classification system display a clean interface where users can upload X-ray images and instantly view detection results. Each result shows bounding boxes, class labels, and confidence scores generated by the YOLOv8 model. The screens are designed to be simple, intuitive, and visually clear, helping users easily interpret dental findings.

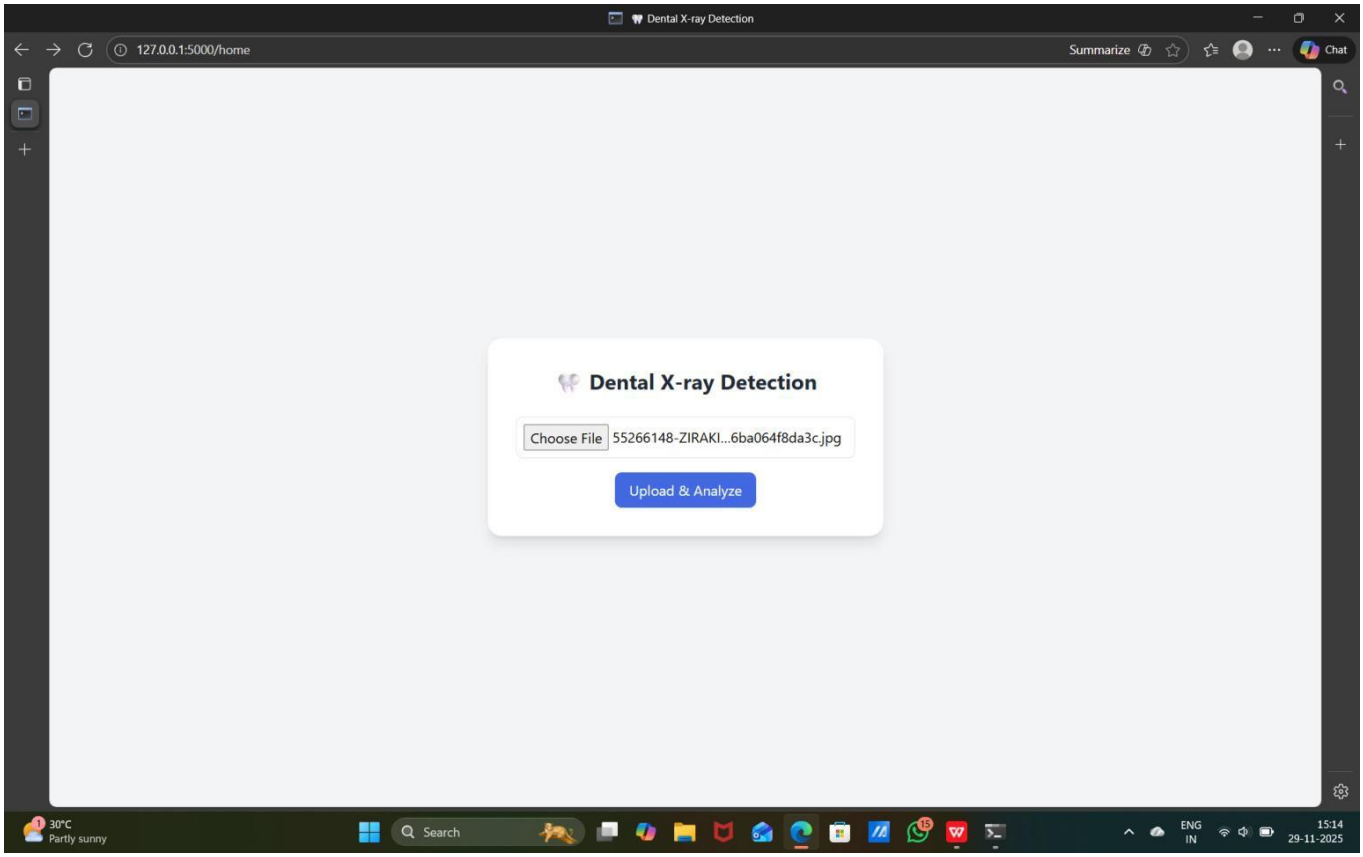

Fig 9: Output Screen

The Dental X-ray Detection page provides a clean and minimal interface focused on uploading dental X-ray images for analysis. At the center, a simple card displays the project title along with a file selection box that allows users to choose an X-ray image from their device. Below the file input, an “Upload & Analyze” button initiates the YOLOv8-based detection process. The layout is uncluttered with a light background, ensuring full attention on the upload section. Overall, the design is user-friendly, efficient, and tailored to make the detection workflow smooth and straightforward.

Predicted result:

The predicted result screen displays the uploaded dental X-ray along with bounding boxes highlighting detected teeth or dental issues. Each detection is labeled with its class name and confidence score generated by the YOLOv8 model. The interface clearly presents the analyzed output, making it easy for users to understand and interpret the dental findings.

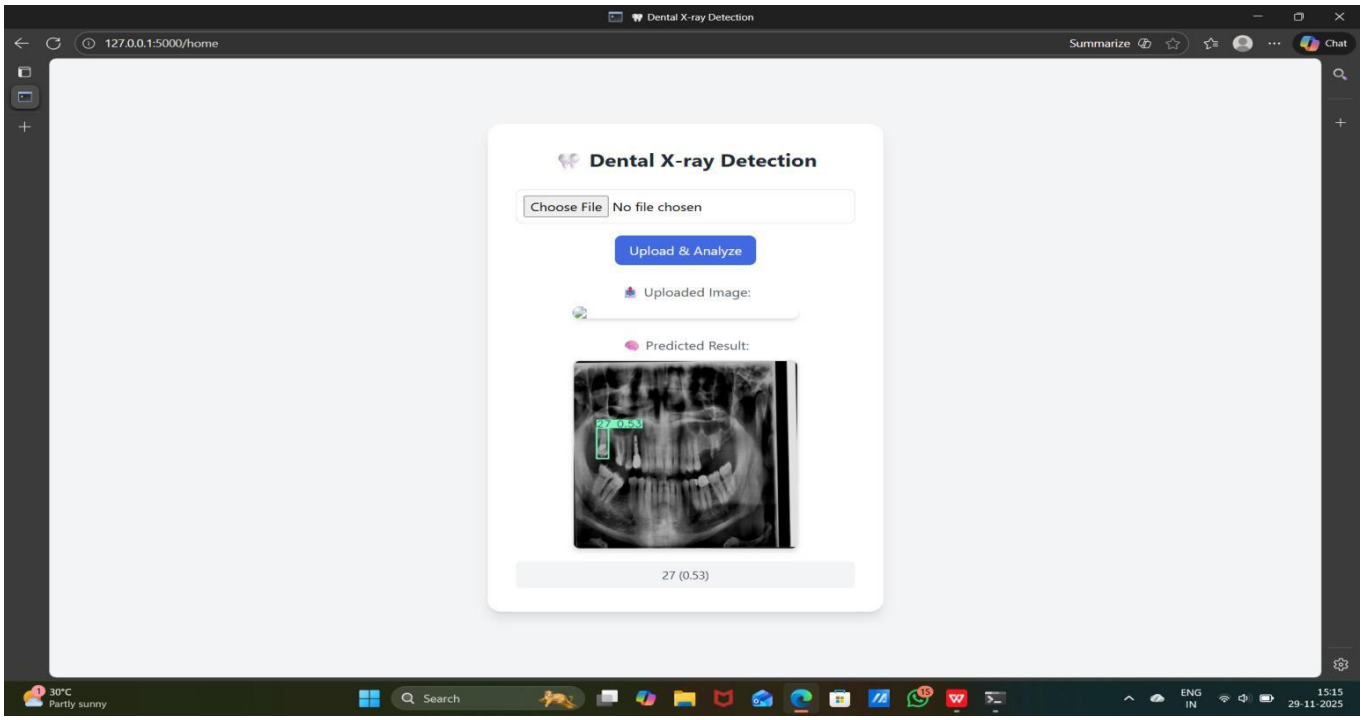

Fig 10: predicted Result

The Predicted Result page clearly displays the uploaded dental X-ray image along with the model's detection output. After selecting a file and clicking "Upload & Analyze," the system processes the image and shows the original upload followed by the analyzed result. The detected tooth or dental condition is highlighted with a green bounding box and labeled with its class ID and confidence score. Below the image, the prediction details are also listed in text format for better clarity. The page design is simple and centered, making it easy for users to review both the uploaded X-ray and the model's interpretation.

Logout page:

The Logout page confirms that the user has successfully signed out of the system and displays a short message thanking them for using the Dental X-ray Image Classification platform. It provides a clean, minimal interface with a button or link to return to the login page for future access. This page ensures secure session termination and prevents unauthorized access after logout.

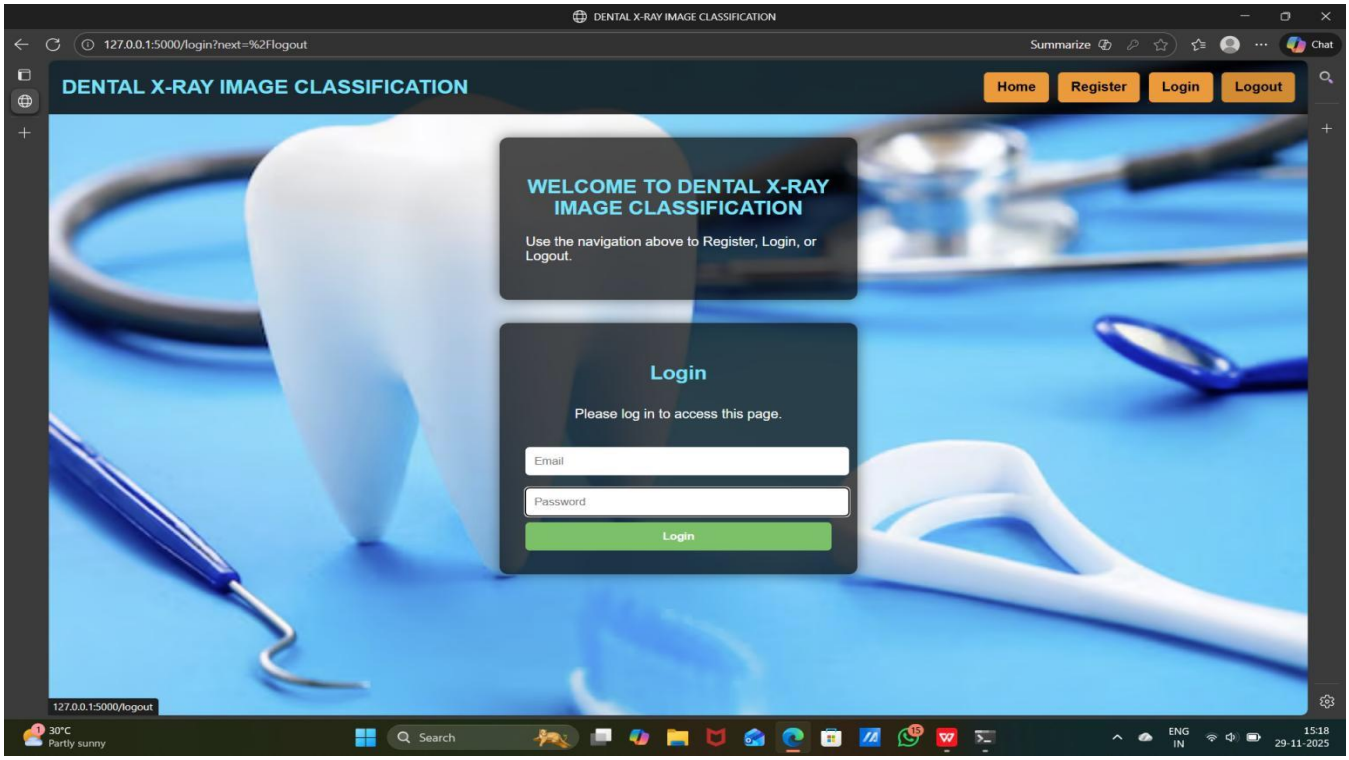

Fig 11: Logout Page

The Logout button in the top-right corner of the interface allows users to securely end their session after using the Dental X-Ray Image Classification system. It is clearly highlighted in orange, making it easy to locate and ensuring smooth navigation for users. When clicked, the button immediately redirects the user to the login page, confirming that the session has been terminated. This prevents unauthorized access by ensuring no one can use the system without logging in again. The Logout feature also helps protect sensitive medical images and user information stored within the system. It is designed to work reliably across different pages so users can safely exit at any time. By integrating proper session handling, the Logout

button enhances both security and usability. Overall, it ensures a trusted and professional workflow for end users.

8.5 Performance Metrics

Table6 – Performance Metrics

Category	**Metric**	**Value**	**Description**
Classification	Accuracy	96%	Percentage of dental X-ray images correctly classified into their respective dental conditions
Detection Quality	Precision	92%	Percentage of predicted dental findings (such as cavities or abnormalities) that are actually correct.
Detection Quality	Recall	89%	Percentage of actual dental issues in the X-ray images that identifies. The model successfully
Generalization	F1-Score	90.5%	Harmonic mean of precision and recall, indicating balanced performance in detecting dental conditions.
Efficiency	Processing Time	0.5s	Average time taken by the model to analyze and classify a single dental X-ray image.

9 Conclusions and Future Scope

9.3 Conclusion

Dental X-ray image classification plays a vital role in enhancing diagnostic accuracy and assisting dental professionals in early detection of oral abnormalities. With the increasing demand for precise and efficient diagnosis, machine learning and deep learning techniques have emerged as powerful tools for automating the interpretation of dental radiography. By leveraging advanced models such as Conventional Neural Networks (CNN s) and YO LO-based architectures, the system can effectively analyze tooth structures, detect lesions, and identify various dental conditions with improved reliability.

The performance of such systems is typically evaluated using metrics like accuracy, precision, recall, F1-score, and IOU, which collectively measure the model's diagnostic capability. One of the major challenges in dental imaging is handling variations in image quality, anatomical differences among patients, and the presence of overlapping structures. A robust classification system must therefore be trained on high-quality, diverse datasets and optimized through processioning techniques and effective feature extraction to ensure consistent results.

Real-time detection and classification also play a crucial role in clinical applications, enabling dentists to make faster decisions and reduce manual workload. Integrating the model into a deployment framework such as Flask or a web-based interface allows seamless operation within dental clinics and research environments. Moreover, continuous updates and retraining with new data are essential to ensure the system adapts to new imaging styles, equipment variations, and emerging diagnostic requirements.

Overall, dental X-ray image classification using deep learning significantly enhances diagnostic efficiency, minimizes human error, and contributes to more accurate and timely detection of dental issues. As technology advances, such automated systems will continue to evolve, offering improved performance and becoming an integral part of modern dental care.

9.4 Limitations

Despite its importance, dental X-ray image classification faces several limitations that can impact its effectiveness. One major challenge is the variability in image quality. Differences in X-ray machines, patient positioning, exposure settings, and noise can lead to inconsistent images, making it difficult for models to generalize well across all cases. Addressing this issue often requires advanced preprocessing techniques, such as normalization, denoising, or augmentation, which may not always perfectly standardize the data.

Another limitation is the complexity and diversity of dental conditions. Subtle variations in tooth decay, bone loss, or other abnormalities can be difficult for models to detect, especially in early stages. Models trained on limited datasets may miss rare or unusual conditions, reducing overall diagnostic accuracy.

False positives and false negatives also present significant challenges. While the system aims to detect abnormalities accurately, misidentifying healthy teeth as problematic can cause unnecessary concern, whereas missing actual issues may delay critical treatment. Achieving a balance between sensitivity and specificity is complex.

Additionally, real-time analysis in clinical settings requires high computational efficiency to provide timely results without delaying patient care. This demand can strain hardware resources, especially in practices with high patient volumes. Furthermore, patient data privacy and security must be maintained, requiring strict compliance with regulations such as HIPAA or GDPR.

These limitations highlight the need for continuous improvement in dental X-ray classification systems, combining advancements in machine learning, regular model updates, and robust infrastructure to address evolving challenges effectively.

9.5 Future Enhancements

1. Incorporating Advanced Machine Learning Techniques:

• Deep Learning Models: Use advanced neural networks, such as Conventional Neural Networks (CNN s), Vision Transformers (Vets), or hybrid models, to capture complex patterns and subtle anomalies in dental X-rays.

• Federated Learning: Enable collaborative model training across multiple dental clinics or hospitals without sharing sensitive patient data, ensuring privacy and compliance with regulations.

.Re2al-Time X-ray Analysis

• Improve systems to provide instant diagnostic insights during dental exams using streaming data frameworks or edge computing for faster processing.

3. Explainable AI (XAI)

• Build explainable models to interpret why certain teeth or regions are flagged as abnormal, improving transparency and trust for dentists and patients.

4. Behavioral and Clinical Analytic s

• Incorporate patient-specific patterns, such as age, oral hygiene habits, previous dental history, and lifestyle factors, to enhance detection accuracy and personalized diagnosis.

5. Integration with Digital Dental Records

• Link classification systems with electronic dental records (EDRs) to create comprehensive, immutable patient histories and improve longitudinal analysis.

5. Enhanced Feature Engineering

• Add dynamic image features, such as tooth alignment angles, bone density, and cavity progression, to capture subtle variations.

• Utilize external data, like patient medical history or radiologist annotations, for richer insights.

6. Dynamic Model Updates

• Implement automated systems to retrain models with new X-rays and updated clinical findings, adapting to emerging dental conditions or imaging trends.

7. Multi-Layered Diagnostic Support

• Combine AI classification with additional tools like 3D imaging, intraoral scans, or biopsy results for more robust and accurate diagnosis.

8. Cross-Modal Imaging Analysis

• Integrate data from multiple imaging sources (e.g., panoramic, periapical, and CBCT scans) to improve detection of complex dental anomalies.

9. Collaborative Dental Databases

• Develop global or regional dental image repositories where anonymized X-rays and findings are shared to improve AI robustness and generalization across diverse populations.

10 Reference

[1] J. E. Cejudo, A. Chaurasia, B. Feldberg, J. Krois, and F. Schwendicke, "Classification of dental radiographs using deep learning," *Journal of Clinical Medicine*, vol. 10, no. 7, p. 1496, 2021.

[2] M. A. Hasnain, S. Ali, H. Malik, M. Irfan, and M. S. Maqbool, "Deep learning-based classification of dental disease using X-rays," *Journal of Computing & Biomedical Informatics*, 2023.

[3] "Periapical dental X-ray image classification using deep neural networks," *PubMed*, includes models such as AlexNet and ResNet-18/34 for classifying healthy and non-healthy periapical X-ray images.

[4] "Deep learning-based recognition of periodontitis and dental caries in dental X-ray images," *PubMed*, utilizes pre-trained EfficientNet-B0 for single-tooth caries and periodontitis detection.

[5] "Instance segmentation and teeth classification in panoramic X-rays," *arXiv*, proposes a U-Net + YOLOv8-based pipeline for segmentation and classification of teeth.

[6] "A systematic review on caries detection, classification, and segmentation from X-ray images: Methods, datasets, evaluation, and open opportunities," *PubMed*, provides a review of approximately 42 computational studies on dental radiograph analysis.

[7] "Application of deep learning in isolated tooth identification," *SpringerLink*, focuses on deep learning-based identification and classification of isolated tooth images.

11 Appendices

Appendices:

• Appendix A: Dataset Description

• Features:

o Image ID: Unique identifier for each X-ray image.

o Patient ID: Unique identifier for the patient.

o Tooth/Region: Specific tooth or region in the X-ray image.

o Image Type: Periapical, panoramic, bitewing, or CBCT scan.

o Image Resolution: Dimensions and quality of the X-ray image.

• Label:

o 0 (Healthy), 1 (Caries), 2 (Periodontitis), 3 (Other dental conditions) — depending on classification type.

• Appendix B: Performance Metrics

• Accuracy: Percentage of correctly classified X-ray images.

• Precision: Fraction of predicted abnormal teeth/regions that are actually abnormal.

• Recall (Sensitivity): Fraction of actual abnormal teeth/regions correctly detected.

• F1-Score: Harmonic mean of precision and recall.

• AUC-ROC: Area under the ROC curve, measuring the ability of the model to distinguish between classes.

• Appendix C: Algorithms Used

• Supervised Learning: Convolutional Neural Networks (CNN), ResNet, EfficientNet, DenseNet.

• Unsupervised Learning: Autoencoders, k-Means Clustering for anomaly detection.

• Hybrid Models: Combination of segmentation (e.g., U-Net) and classification models (e.g., CNN, YOLOv8).

• Appendix D: Tools and Libraries

• Programming Languages: Python.

• Libraries:

o TensorFlow, PyTorch: Model building and training.

o OpenCV, PIL: Image preprocessing and augmentation.

o NumPy, pandas: Data handling and feature extraction.

o Flask or FastAPI: Deployment for real-time dental X-ray classification.

• Appendix E: Real-Time Setup

• Data Ingestion: Streaming frameworks or batch pipelines for X-ray uploads (e.g., DICOM servers).

• Deployment Frameworks: TensorFlow Serving, TorchServe, Flask, or FastAPI.

• Hardware: GPU-enabled servers for fast inference during clinical exams.

• Appendix F: Ethical Considerations

• Privacy: Ensure compliance with HIPAA, GDPR, or local medical data protection regulations.

• Bias Mitigation: Address biases in the dataset to ensure equitable diagnosis across age, gender, and ethnicity.

• Explainability: Provide interpretable results for dentists to trust AI-based classifications.